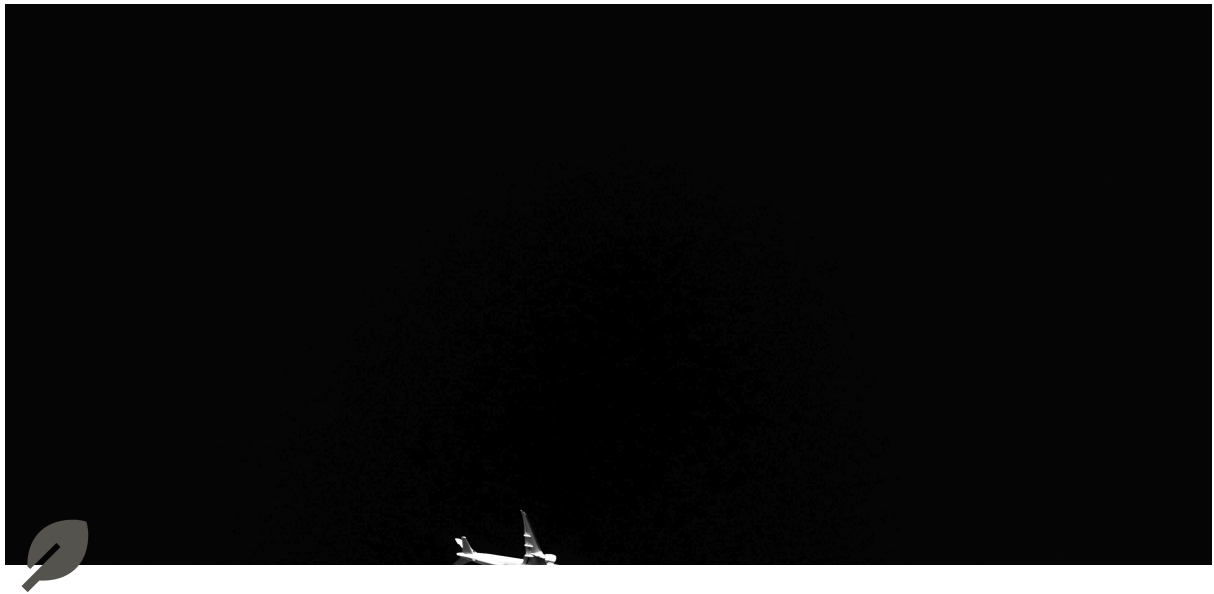




Práctica Ambiente de Trabajo

[2026-02 AmbienteTrabajo.pdf](#)

[2020-01 Ejercicios Linux.pdf](#)

1. Parte A

[1.1. "Crear el ambiente de trabajo inicial con una máquina cliente y una máquina servidor"](#)

[1.2. "Instalar net-tools y vim"](#)

[1.3. "Publicar Box a Vagrant Cloud"](#)

[1.4. "Realizar el taller de Linux"](#)

[1.5. "Directorios Sincronizados en Vagrant"](#)

2. Parte B

[2.1. "Cómo funcionan los repositorios en git"](#)

[2.2. "Flujo para crear un repositorio de git y sincronizarlo con Github \(DESDE MÁQUINA VIRTUAL\)"](#)

[2.3. "Github Tokens"](#)

1. Parte A



1.1. "Crear el ambiente de trabajo inicial con una máquina cliente y una máquina servidor"

1. **Iniciar Vagrant:** Con el comando `vagrant init`, se crea un Vagrantfile base, el cual se editará más adelante.
2. **Configurar Vagrantfile:** Se configura el vagrant file (con vim) con configuración básica que incluye: dos máquinas virtuales, con su respectiva box de **bento/ubuntu-22.04** (desde Vagrant Cloud), IP privada, y hostname.
 - a. `config.vm.define :<nombre-maquina> do |nombre-maquina|`: Configurar una nueva máquina virtual
 - b. `<nombre-maquina>.vm.box`: Asignar una box de Vagrant Cloud para dicha máquina virtual (Una box, es una máquina ya lista para correr). En este caso, una con **Ubuntu-22.04**
 - c. `<nombre-maquina>.vm.network :private_network, ip: "<ip>"`: Sirve para configurarle una dirección ip puntual a la máquina virtual en IPv4.
 - d. `<nombre-maquina>.vm.hostname = "<nombre>"`: Asigna un nombre a la máquina virtual en el sistema operativo.

```
# -*- mode: ruby -*-
# vi: set ft=ruby :

Vagrant.configure("2") do |config|
  config.vm.define :servidor do |servidor|
    servidor.vm.box = "bento/ubuntu-22.04"
    servidor.vm.network :private_network, ip: "192.168.50.3"
    servidor.vm.hostname = "servidor"
  end

  config.vm.define :cliente do |cliente|
    cliente.vm.box = "bento/ubuntu-22.04"
    cliente.vm.network :private_network, ip: "192.168.50.2"
    cliente.vm.hostname = "cliente"
  end
end
```

Posible pregunta: ¿Cuál es una ventaja de crear las máquinas virtuales con Vagrant?

A diferencia de si se crearan manualmente, una de las ventajas es que se puede configurar todo en un archivo por medio de código para automatizar su proceso, además que no es necesario descargar imágenes ISO para configurar un SO sino que basta con vincularlo con una box existente, ya sea local o de Vagrant Cloud.

3. **Iniciar las máquinas:** Se crean y se inician las máquinas virtuales con base a la configuración en el Vagrantfile con el comando `vagrant up`

4. **Verificar el estado de las máquinas:** Usando el comando `vagrant status` se puede ver el estado actual de las máquinas (en este caso **running**)

```
[~/Documents/Servicios Telmáticos/Practica Ambiente de Desarrollo]
eddie> vagrant status
Current machine states:

servidor                running (virtualbox)
cliente                 running (virtualbox)

This environment represents multiple VMs. The VMs are all listed
above with their current state. For more information about a specific
VM, run `vagrant status NAME`.
```

▼ 1.2. “Instalar net-tools y vim”

1. **Entrar en cada máquina virtual:** Para ello se utiliza el comando `vagrant ssh <nombre-máquina>`. Esto lo que hace es iniciar una conexión SSH para entrar a la máquina
2. **Autenticarse como superusuario:** Se realiza este procedimiento para poder hacer las instalaciones sin necesidad del `sudo`, **para entrar como superusuario se usa `sudo -i`**
3. **Instalar los paquetes:** Se instalan lo requerido con el gestor de paquetes apt: `apt install <nombre-paquete>`

```
apt install net-tools vim
```

4. **Visualizar configuración de red:** Para comprobar la configuración de red de una máquina virtual se usa el comando `ifconfig`.
 - a. Hay dos configuraciones una en `eth0` y otra en `eth1`
 - i. **`eth0`:** Es la configuración de la interfaz con la que la máquina virtual sale a internet
 - ii. **`eth1`:** Es la configuración de la interfaz configurada en el `Vagrantfile` y que permite la conexión entre las máquinas por medio del ping.
5. **Comunicación entre las máquinas virtuales:** Desde el cliente o el servidor se hace un ping a sus respectivas IP's.
4. **Salir de la máquina virtual:** Para cerrar la conexión con la máquina virtual, se escribe en la terminal `exit` y se presiona **Ctrl + D**.

▼ 1.3. "Publicar Box a Vagrant Cloud"

1. **Empaquetar box:** Una de las funcionalidades que ofrece vagrant es poder subir a la nube nuestras propias boxes para utilizarlas más fácilmente.

Ese .box contiene el sistema operativo y todo el software que quedó instalado y configurado en la máquina virtual.

```
vagrant package <nombre_maquina> --out <nombre_box>.box
```

2. **Agregar box localmente:** Para que dicho box pueda ser empleado localmente en las próximas instalaciones, debe de añadirse:

```
vagrant box add <nombre_box> <nombre_box>.box
```

3. **Comprobar instalación:** Para visualizar todos los boxes del catálogo local se usa:

```
vagrant box list
```

4. **Agregar un nuevo box en Vagrant Cloud:** Una vez iniciado sesión en Vagrant Cloud, ir al apartado de *Vagrant Registries* y crear una nueva box:
https://portal.cloud.hashicorp.com/services/vagrant/registries?project_id=f37d5f32-48e9-4db2-9ddf-e7aa20617799

- Posterior a la creación básica:
 - Se crea una versión de la box
 - Agregar como provider a VirtualBox y subir el archivo .box. Se debe seleccionar la arquitectura **amd64**.
 - Una vez cargado el box, se debe presionar el botón **Release** para lanzarlo y que pueda ser usado.

▼ 1.4. "Realizar el taller de Linux"

1. Listar el directorio actual

- **pwd**: Lista la ruta en la que se encuentra ubicado en el momento.

2. Volver al directorio home desde cualquier ubicación

- **cd**: Sirve para navegar entre directorios
 - **cd [carpeta]**: Navega hacia esa carpeta (hacia delante)
 - **cd ..**: Regresa a la carpeta anterior
 - **cd**: Regresa al directorio home

3. Crear un nuevo directorio

- **mkdir [nombre_carpeta]**: Crea una carpeta en la ruta actual
- **mkdir -p**: Permite crear todos los directorios padre que no existe.
 - **Ejemplo**: **mkdir -p temp/stuff/things/orange/apple/pear/grape**.
 - La idea es crear un directorio **grape** pero en el path si alguno de los directorios anteriores no existen, se crean automáticamente.

5. Listar los contenidos de un directorio:

- **ls**: Sirve para listar el contenido de un directorio (subdirectorios o archivos)

6. Eliminar un directorio vacío

- **rmdir [nombre_carpeta]**: Permite eliminar un directorio siempre y cuando esté vacío.

7. Localización temporal (pushd, popd):

- Son comandos para moverse entre directorios de forma temporal y guardar la dirección actual
 - **pushd [ruta]**: Permite moverse a una ruta en concreto, pero guardando mi ubicación actual para retornar rápidamente.
 - **popd**: Sirve para regresar a la dirección que quedó guardada con pushd
- Esto funciona tal como una pila.

8. Crear archivos vacíos:

- **touch [nombre_archivo].[extensión]**: Crea un archivo vacío

8. Copiar un archivo

- **cp [nombre_archivo] [ruta_copia]**: Copia un archivo a una ruta concreta.
- **cp -r [nombre_directorio] [ruta_copia]**: Copia un directorio completo junto su contenido a una ruta concreta.

8. Mover y renombrar un archivo o directorio

- **mv [nombre_archivo] [nuevo_nombre]**: Renombra un archivo o directorio
- **mv [nombre_archivo] [nueva_ruta]**: MUEVE un archivo o directorio hacia la nueva ruta.

8. Less y more (Diferencia)

- Son comandos que sirven para ver el contenido de un archivo largo página por página.

Característica	<code>more [archivo]</code>	<code>less [archivo]</code>
Avanzar página	✓	✓
Retroceder página	⚠ Limitado (Es permitido solo en versiones recientes)	✓
Buscar texto (<code>/texto</code>)	⚠ Limitado	✓ (Más funciones de búsqueda)
Rapidez	Más lento	Más rápido

- **Controles**

- Espacio: Página siguiente
- Enter: Ver una línea más
- b: Página anterior (solo para less)
- /texto: Buscar una palabra
- q: Salir

- Se recomienda usar más less

8. Ver contenido de archivo:

- `cat [nombre_archivo]`: Permite ver el contenido **COMPLETO (sin paginación)** de un archivo

8. Eliminar un archivo:

- `rm [nombre_archivo]`: Permite eliminar un archivo en la ubicación dada.
- `rm [archivo_1] [archivo_2]`: Permite eliminar varios archivos de golpe.

▼ 1.5. “Directorios Sincronizados en Vagrant”

- Un directorio sincronizado es una carpeta que existe **tanto entre el host y la máquina virtual** y cuyo **contenido se mantiene sincronizado automáticamente**.

Todo lo que se guarde en la carpeta desde el host, aparecerá inmediatamente dentro de la máquina virtual y viceversa.

- Sirve por ejemplo para intercambiar archivos rápidamente.
- El directorio sincronizado se crea automáticamente al ejecutar **vagrant up** y es la **carpeta donde se encuentra el Vagrantfile**.
 - Todos los archivos que estén ahí se compartirán automáticamente (sin necesidad de reiniciar las máquinas)
- Para acceder a este directorio sincronizado desde la máquina virtual se debe de acceder al **directorio llamado vagrant** desde la raíz:

```
cd /vagrant
ls
```

1.5.1 Agregar directorios sincronizados

Para añadir más directorios sincronizados se hace desde el Vagrantfile, por ejemplo globalmente de la siguiente forma:

```
# SINTAXIS
config.vm.synced_folder "carpeta_host", "carpeta_guest"

# EJEMPLO - 2 DIRECTORIOS SINCRONIZADOS ADICIONALES AL DEFAULT
config.vm.synced_folder "./codigo", "/var/www"
config.vm.synced_folder "./frontend", "/frontend"
```



Pueden haber tantos directorios sincronizados se deseen.

Resumen: Básicamente los directorios sincronizados hace referencia a carpetas vinculadas automáticamente entre la máquina host y la máquina virtual.

2. Parte B

▼ 2.1. "Cómo funcionan los repositorios en git"

- Git es un **sistema de control de versiones** que nos permite llevar un **registro de los cambios** realizados en repositorio concreto a forma de línea de tiempo.
- Un repositorio, es el lugar donde se guardan los archivos del proyecto, el historial de cambios, información de quién hizo cada cambio, ramas, etc.
 - Es decir, un repositorio no es simplemente una carpeta donde se guardan los archivos
- Git vive localmente en la carpeta `.git`, que se crea automáticamente al ejecutar **git init** en un directorio.

2.1.1 Flujo de Funcionamiento

1. Crear un archivo base
2. Cuando se ejecuta **git add hola.txt**, git pasa ese archivo a la **staging area** (o preparación), básicamente aquí están los archivos listos para ser commiteados.
3. Cuando se ejecuta **git commit -m ""**, se crea un nodo en la línea temporal de cambios en git con referencias a los archivos modificados y se le calcula un hash único que es el **identificador del commit**.



2.2. "Flujo para crear un repositorio de git y sincronizarlo con Github (DESDE MÁQUINA VIRTUAL) "

1. Crear repositorio desde github
2. Entrar a la máquina virtual por ssh (**vagrant ssh [maquina]**)
3. **Instalar git en la máquina**

- a. Autenticarse como superusuario: **sudo -i**
- b. Instalar git: **apt install git**

4. Configurar usuario de git:

```
git config --global user.email "tuemail@gmail.com"
git config --global user.name "tuusername"
```

5. Crear una carpeta para el repositorio y acceder a ella **(se recomienda hacerlo en el directorio sincronizado /vagrant)**
6. Ejecutar **git init**
7. Cambiar el nombre de rama máster a main: **git branch -m main**
8. Crear archivo y modificarlo
9. Añadir archivo a la **staging area**: **git add [archivo]**
10. Hacer un commit: **git commit -m "mensaje"**
11. Añadir la conexión remota con el repositorio de github: **git remote add origin "url_github"**
12. **Configurar Github Token (Ver 2.3.)**
13. Subirlo a github: **git push -u origin main**

▼ 2.3. "Github Tokens"

Para poder utilizar y modificar un repositorio de Github, se requiere crear un Token por cada una de ellas, siguiendo los siguientes pasos:

1. **Generar una clave SSH:** Aceptar todas las preguntas durante la generación.

```
ssh-keygen -t ed25519 -C "tu_correo@example.com"
```

2. **Iniciar el agente SSH:**

```
eval "$(ssh-agent -s)"
```

3. **Agregar la clave al agente:**

```
ssh-add ~/.ssh/id_ed25519
```

4. **Copiar la clave pública**

```
cat ~/.ssh/id_ed25519.pub
```

5. **Agregar la clave a Github**

- a. En Github, ir a Settings
- b. SSH and GPG keys
- c. New SSH key
- d. Escribir un nombre (por ejemplo, Ubuntu VM)
- e. Pegar la clave pública
- f. Hacer click en Add SSH key

6. **Verificar la conexión:** Al escribir el comando, escribir **yes**

```
ssh-Tgit@github.com
```